

```
#!/usr/bin/env python3
```

```
"""
```

Runtime Pipeline Server

Source Origin: PROPOSED WORKFLOWS AND FILE STRUCTURES / NovA-Claw.py

Target Subsystem: novaexopia / aesc / runtimes

Provides an asynchronous WebSocket daemon (port 8765) bridging mobile UI clients (Horizons UI / launcher_ui) to local LLM inference engines (Hexagon NPU / llama.cpp) with cloud fallback, local ADB tool execution, and episodic memory persistence.

```
"""
```

```
import asyncio
```

```
import json
```

```
import os
```

```
import subprocess
```

```
import logging
```

```
from typing import Dict, Any, Optional
```

```
logging.basicConfig(level=logging.INFO, format="[%(asctime)s] %(levelname)s: %(message)s")
```

```
class MemoryPlatformBridge:
```

```
    def __init__(self, storage_path: str = "aesop_memory.json"):
```

```
        self.storage_path = storage_path
```

```
        if not os.path.exists(self.storage_path):
```

```
            with open(self.storage_path, "w") as f:
```

```
                json.dump({"session_parameters": "AESOP-XI", "history": []}, f)
```

```
    def retrieve_context(self, prompt: str) -> str:
```

```
        try:
```

```
            with open(self.storage_path, "r") as f:
```

```
                data = json.load(f)
```

```
            history = data.get("history", [])
```

```
            if history:
```

```
                last_output = history[-1].get("output", "")
```

```
                return f"Active Platform Layer: AESOP-XI. Last verified device state: {last_output}"
```

```
        except Exception as e:
```

```
            logging.warning(f"Memory read fault: {e}")
```

```
            return "Active Platform Layer: AESOP-XI. Clean slate initialization."
```

```
    def commit_state(self, prompt: str, output: str):
```

```
        try:
```

```
            with open(self.storage_path, "r+") as f:
```

```
                data = json.load(f)
```

```

        data.setdefault("history", []).append({"prompt": prompt, "output": output})
        f.seek(0)
        json.dump(data, f, indent=2)
        f.truncate()
    except Exception as e:
        logging.error(f"Memory commit fault: {e}")

def adb_change_wallpaper(color_hex: str) -> str:
    """Executes ADB broadcast to alter UI wallpaper/accent color via local loopback."""
    cmd = f"adb shell am broadcast -a com.horizons.ui.UPDATE_BG --es color '{color_hex}'"
    try:
        res = subprocess.run(cmd, shell=True, capture_output=True, text=True, timeout=5)
        return "UI repaint success" if res.returncode == 0 else f"ADB Fault: {res.stderr.strip()}"
    except Exception as e:
        return f"ADB Execution Exception: {e}"

AVAILABLE_TOOLS = {
    "adb_change_wallpaper": adb_change_wallpaper
}

TOOLS_SCHEMA = [
    {
        "type": "function",
        "function": {
            "name": "adb_change_wallpaper",
            "description": "Changes Android background/accent color via ADB loopback broadcast.",
            "parameters": {
                "type": "object",
                "properties": {
                    "color_hex": {"type": "string", "description": "Hex color string (e.g. #1E1E2E)"}
                },
                "required": ["color_hex"]
            }
        }
    }
]

```

```
memory_bridge = MemoryPlatformBridge()
```

```

async def handle_client_socket(websocket, path=None):
    logging.info("Client connected to Runtime Pipeline Server.")
    async for message in websocket:
        try:

```

```

payload = json.loads(message)
user_prompt = payload.get("prompt_text", "")
context = memory_bridge.retrieve_context(user_prompt)
logging.info(f"Received prompt: {user_prompt[:80]}")

# Execution simulation / bridge logic
response_text = f"Processed prompt: '{user_prompt}' under context: {context}"
await websocket.send(json.dumps({
    "action": "DISPLAY_TEXT",
    "data": response_text
}))
memory_bridge.commit_state(user_prompt, response_text)
except Exception as e:
    logging.error(f"Socket handling error: {e}")
    await websocket.send(json.dumps({"action": "ERROR", "message": str(e)}))

async def main():
    host = "0.0.0.0"
    port = 8765
    logging.info(f"Starting Runtime Pipeline Server on ws://{host}:{port}...")
    server = await websockets.serve(handle_client_socket, host, port)
    await server.wait_closed()

if __name__ == "__main__":
    try:
        import websockets
        asyncio.run(main())
    except ImportError:
        logging.warning("websockets library not installed. Script configured as executable
standalone reference.")

```